DocGen_Architecture_and_Usage

DocGen — Solution Architecture and Usage

Package: Portwood DocGen Managed **Namespace:** `portwoodglobal` **Version:** 2.1.0 **Package Version Id:** `04tVx000000Zw5xIAC` **Released:** Yes (promoted 2026-05-22) **Prior listing version (AppExchange):** v1.56.0 (`04tal000006i1rNAAQ`) — this submission addresses all 30 findings from the AppExchange security review against the v1.56 listing. v2.0/v2.1.0 also rolls forward ~45 versions of feature work since v1.56.

Install URLs:

- **Production:** https://login.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000Zw5xIAC
- **Sandbox:** https://test.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000Zw5xIAC
- **CLI:** `sf package install --package 04tVx000000Zw5xIAC --wait 10 --target-org <your-org>`

> Companion to `DocGen_Solution_Architecture_and_Usage.md`, which focuses on AppExchange security review. This document describes *what* the solution is, *how it is put together*, and *how a customer uses it* day to day.

---

1. What DocGen Does

DocGen is a 100% native Salesforce document generation engine. It lets admins upload ordinary Microsoft Office templates (`.docx`, `.xlsx`, `.pptx`) containing merge-tag placeholders, and end users generate finished documents from any Salesforce record — as PDF, DOCX, XLSX, or PPTX — without ever leaving the Salesforce platform.

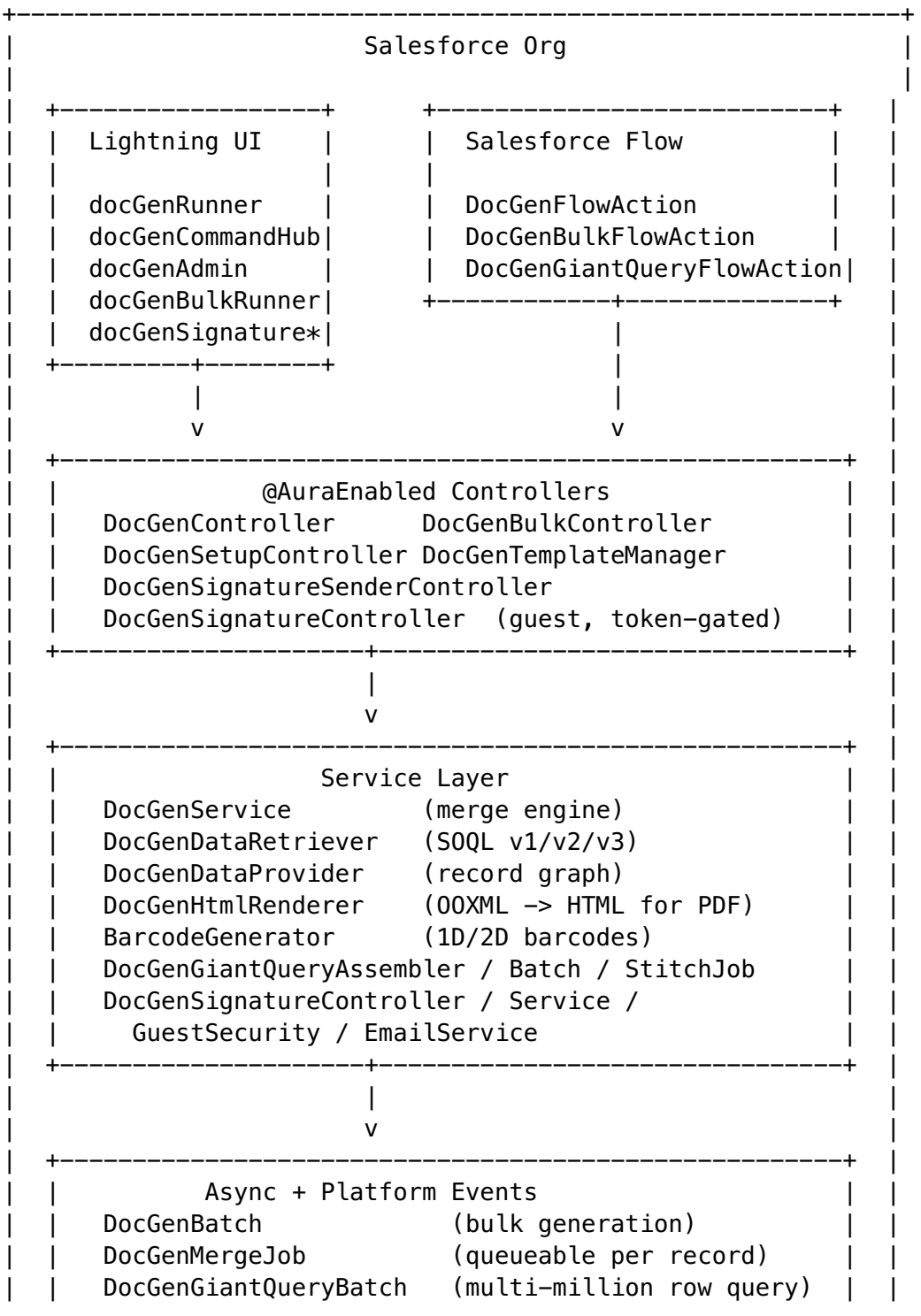
Capabilities in v2.1.0:

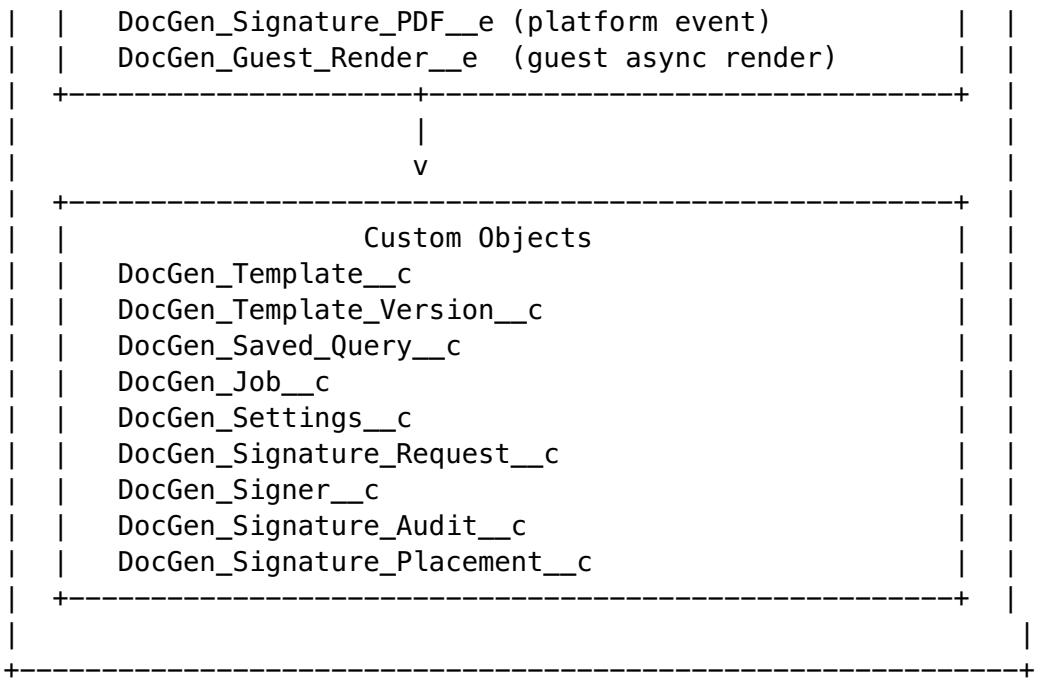
- Document generation from any standard or custom object.
- Multi-object query trees (parent, child, and junction relationships) at arbitrary depth.
- Merge tags for fields, loops, conditionals, images, and barcodes.
- PDF rendering via Salesforce's built-in `Blob.toPdf()` service.
- Client-side DOCX/XLSX/PPTX assembly for unlimited-size outputs.

- Bulk generation against saved queries, with Flow invocable actions.
- Built-in electronic signatures (Signatures v3 — typed name + email PIN, optional PIN bypass / in-person signing, sequential or parallel signing order, per-tag guided placements) with audit trail and document verification.
- Salesforce Flow integration (invocable actions for single and bulk generation).
- Native Command Hub with visual template builder, query builder, and job history.

---

2. High-Level Architecture





Everything inside the box runs inside the customer's Salesforce org. Nothing crosses the org boundary.

---

3. Component Inventory

3.1 Custom Objects

Object	Purpose
`DocGen_Template__c`	Logical template: name, base object, status, default output format.
`DocGen_Template_Version__c`	Versioned template artifact (holds the uploaded OOXML via CV).
`DocGen_Saved_Query__c`	Reusable query definition (V1 / V2 / V3 formats).
`DocGen_Job__c`	Bulk-generation job tracking (counts, status, error summary).
`DocGen_Settings__c`	Hierarchy custom setting for org-level configuration.
`DocGen_Signature_Request__c`	Parent record for a signature request.
`DocGen_Signer__c`	One per signer (token, PIN hash, status, typed name).
`DocGen_Signature_Audit__c`	Immutable audit record (IP, UA, hash, timestamps, history tracking).
`DocGen_Signature_Placement__c`	Per-tag guided placement metadata for Signatures v3.
`DocGen_Signature_PDF__e`	Platform event that triggers async PDF finalization.

Object	Purpose
DocGen_Guest_Render__e	Platform event used by guest-rendering async path.

Custom-object count grew from 9 (v1.56) to 11 in v2.0 — DocGen_Signature_Placement__c and DocGen_Guest_Render__e were added during the rollforward.

3.2 Apex Classes

v2.1.0 ships **42 non-test Apex classes** and **29 test classes** (v1.56 baseline: 27 / 19; v2.0 baseline: 41 / 28 — v2.1.0 adds DocGenFlsGuard + DocGenFlsGuardTest).

Controllers (@AuraEnabled, with sharing):

- DocGenController — primary entry point for docGenRunner and docGenAdmin. Generation, template CRUD, metadata probes.
- DocGenBulkController — bulk job creation, progress polling, analysis.
- DocGenSetupController — setup wizard + Command Hub metadata.
- DocGenTemplateManager — template library and version management.
- DocGenSignatureSenderController — admin-initiated signature requests.
- DocGenAuthenticatorController — setup/permission check helper; the v2.0 verifyDocument now returns List<VerificationResult> so multi-signer audit trails surface in full (see §13).

Controllers (guest-facing, token-gated, without sharing):

- DocGenSignatureController — guest signing page entry point (validation + capture).

Service layer:

- DocGenService — the merge engine. Decompresses OOXML, walks processXml(), replaces tags, rebuilds ZIP. Owns the currentOutputFormat heap-skipping trick for PDFs.
- DocGenDataRetriever — SOQL executor with V1 (legacy flat string), V2 (flat JSON + junctions), V3 (query-tree) config support.
- DocGenDataProvider — record-graph helpers used during merge.
- DocGenHtmlRenderer — converts merged OOXML to HTML for Blob.toPdf().
- BarcodeGenerator — Code-128 / QR generation for barcode merge tags.
- DocGenGiantQueryAssembler / DocGenGiantQueryBatch / DocGenGiantQueryStitchJob — multi-million-row query pipeline for bulk jobs.
- DocGenSignatureController — validation + capture at the guest endpoint.
- DocGenSignatureService — Queueable PDF generation, runs as Automated Process during signature finalization.
- DocGenSignatureGuestSecurity — centralized Schema-CRUD describe checks + token-bound capability validation + per-operation field allowlists for every guest entry point (new in v2.0).
- DocGenFlsGuard — per-field FLS describe-check helper (assertCreateable/assertUpdateable/assertAccessible). Called immediately before every package-namespaced Database.<op>(...,

`AccessLevel.SYSTEM_MODE)` DML and every `WITH SYSTEM_MODE` SOQL. 243 call sites across 19 controllers (70 DML + 173 SOQL). NEW in v2.1.0.
- `DocGenSignatureEmailService` — branded email dispatch for signers.

Async:

- `DocGenBatch` — Batchable bulk-generation driver.
- `DocGenMergeJob` — Queueable per-record merge.
- `DocGenGiantQueryBatch` — Batchable over saved query shards.

Flow invocables:

- `DocGenFlowAction` — single-record document generation.
- `DocGenBulkFlowAction` — bulk generation against a saved query.
- `DocGenGiantQueryFlowAction` — trigger a giant-query job.
- `DocGenSignatureFlowAction` — create a DocGen signature request from a Flow and return one signing URL per signer. Defaults to silent (no package-sent emails) so the Flow author owns the notification path via Send Email / Slack / custom invocable.

3.3 Visualforce Pages

4 VF pages (unchanged from v1.56):

- `DocGenGuide.page` — in-app admin guide.
- `DocGenSign.page` / `DocGenSignature.page` — the guest signing page (served by a Salesforce Site).
- `DocGenVerify.page` — public verification page; computes SHA-256 locally in the browser.

3.4 Lightning Web Components

v2.0 ships **18 LWC bundles** (v1.56 baseline: 17).

- `docGenRunner` — record-page component. Generates and downloads (or saves) documents.
- `docGenCommandHub` — the DocGen app landing page (quick actions, template library, bulk runner, help).
- `docGenAdmin` — template CRUD and versioning UI.
- `docGenSetupWizard` — first-run setup wizard.
- `docGenAdminGuide` — embedded admin documentation.
- `docGenQueryBuilder` — legacy flat query builder (manual SOQL).
- `docGenColumnBuilder` + `docGenTreeBuilder` + `docGenTreeNode` — V3 visual query tree builder with tab-per-object layout.
- `docGenFilterBuilder` — WHERE-clause builder.
- `docGenBulkRunner` — bulk job launcher + progress.
- `docGenSignatureSender` — admin component to invite signers from a record page.
- `docGenSignatureSettings` — branding + OWA configuration.
- `docGenSharing` — sharing-rule management helper.
- `docGenTitleEditor` / `docGenAuthenticator` / `docGenUtils` — small helpers.

Shared JS modules inside `docGenRunner`:

- `docGenZipWriter.js` — pure-JS, dependency-free ZIP writer (store mode, CRC-32). Produces valid DOCX/XLSX/PPTX in the browser.
- `docGenPdfMerger.js` — helper used when assembling multi-section PDFs.

3.5 Permission Sets

v2.0 ships **4 permission sets** (v1.56 baseline: 3).

Permission Set	Who	Scope
`DocGen_Admin`	Admins	Full template CRUD, bulk jobs, settings, signature requests.
`DocGen_User`	End users	Run generation from record pages, view own jobs, view templates.
`DocGen_Guest_Signature`	Site guest user	Read-only on signature objects, exclusively through token-gated entry points.
`DocGen_Guest_Runner`	Site guest user	Guest-side document-generation paths (token-gated); added since v1.56.

3.6 Tabs

`DocGen Command Hub`, `DocGen Template Manager`, `DocGen Bulk Gen`, `DocGen Admin Guide`, `DocGen Setup`, plus object tabs for `DocGen_Job__c`, `DocGen_Template__c`, `DocGen_Template_Version__c`, `DocGen_Signature_Request__c`, `DocGen_Signer__c`, `DocGen_Signature_Audit__c`.

---

4. Data Flow Narratives

4.1 Single-Record Generation

1. User opens a record page with the `docGenRunner` component.
2. Component loads the list of templates whose base object matches the record's sObject type.
3. User picks a template, chooses **Download** or **Save to Record**, and clicks **Generate**.
4. `docGenRunner` calls `DocGenController.processAndReturnDocument(templateId, recordId, outputFormat)`.
5. `DocGenDataRetriever` runs the template's saved query (V1/V2/V3) against the record. Standard objects are read in `WITH USER_MODE`; package-namespaced custom-object reads pass the object-level Schema-CRUD gate first and then issue `SYSTEM_MODE` SOQL (see §5).
6. `DocGenService.mergeTemplate()` loads the active template version, decompresses the OOXML, walks `processXml()` on every XML part, replaces

merge tags, and either rebuilds the ZIP (DOCX path) or emits the HTML the PDF renderer needs (PDF path).
7. **PDF path:** `DocGenHtmlRenderer.convertToHtml()` produces HTML with relative Shepherd image URLs → `Blob.toPdf()` renders the PDF inside the org.
8. **DOCX/XLSX/PPTX path:** either server-side ZIP rebuild (small docs, "Save to Record" under ~4 MB) or client-side assembly via `docGenZipWriter.js` (large docs, download).
9. Output returned to the browser (download) or written as a new `ContentVersion` on the source record.

4.2 Bulk Generation

1. Admin defines a Saved Query (via `docGenBulkRunner` or programmatically) and picks the template.
2. `DocGenBulkController.startBulkJob()` creates a `DocGen_Job__c` record and enqueues `DocGenBatch` (Batchable).
3. Each batch slice runs `DocGenMergeJob` equivalents internally — for each record, generate the merged output and attach it as a `ContentVersion`.
4. Progress is polled by `docGenBulkRunner` via `DocGenBulkController.getJobStatus()`.
5. On completion, the `DocGen_Job__c` record contains per-record status, error summary, and links to generated files.

4.3 Giant-Query Path

For queries that would exceed single-transaction SOQL limits, `DocGenGiantQueryAssembler` splits the work:

1. Admin triggers a giant query (or Flow invocable does) — one record per sharded chunk.
2. `DocGenGiantQueryBatch` (Batchable) walks the shards.
3. `DocGenGiantQueryStitchJob` (Queueable) assembles the results back into a single logical dataset for the template merge.

4.4 Signatures v3 Flow

See `DocGen_Solution_Architecture_and_Usage.md` §2.4 for the security-focused version. In summary:

1. Admin creates a signature request from a record page (`docGenSignatureSender`). v3 supports **sequential or parallel** signing order and **per-tag guided placements** via `DocGen_Signature_Placement__c` records (one per signature tag, so the signer is walked through the document tag-by-tag instead of seeing all tags at once).
2. Each signer gets a unique token + branded email. Admins can elect **PIN bypass / in-person signing** when capturing signatures face-to-face.
3. Signer opens the public `DocGenSignature.page`, completes email-PIN verification (or proceeds directly for in-person mode), reviews the preview, types their name on each placement, checks consent, submits. Every guest call passes through `DocGenSignatureGuestSecurity` for token-bound capability validation and per-operation field allowlists.

4. When the last signer completes, `DocGen_Signature_PDF__e` publishes; `DocGenSignatureService` (Queueable, runs as Automated Process) merges the template, replaces `{@Signature_Role}` placeholders with typed names, appends an Electronic Signature Certificate, generates the final PDF, hashes it (SHA-256), and saves it as a `ContentVersion` on the related record.
5. The hash is also written to the audit record and exposed via `DocGenVerify.page`. In v2.0, dropping a multi-signer PDF on the verifier returns the **complete** audit trail for every signer (the v1.56 implementation returned LIMIT 1 — fixed by the `verifyDocument` → `List<VerificationResult>` change).

---

5. CRUD/FLS Enforcement Model

v2.1.0 uses a three-layer CRUD/FLS pattern across every admin `@AuraEnabled` / `@InvocableMethod` entry point. The v1.56 reviewer's stated finding-resolution language was: *"enforce CRUD checks on the object **AND** FLS checks on the fields before performing any DML operation, **or** alternatively use USER_MODE."* v2.0 implemented the object-level half; v2.1.0 adds the per-field half via the new `DocGenFlsGuard` helper.

1. **Object-level Schema-CRUD gate** at every entry point (shipped in v2.0) — `if (!Schema.sObjectType.<Object>.isAccessible|isCreateable|isUpdateable()) throw new DocGenException('Insufficient access…')`. This is the documented enforcement signal that the AppExchange `sfge:ApexFlsViolation` rule pattern-matches on; it gates the entire method body on the user's DocGen permission-set assignment.

2. **Per-field `Schema.SObjectField.getDescribe().is{Createable,Updateable,Accessible}()` check via `DocGenFlsGuard`** (NEW in v2.1.0) — called immediately before every package-namespaced DML and `WITH SYSTEM_MODE` SOQL. Performs an explicit `getDescribe().is*()` per field in the allowlist; throws `DocGenException` if any field is denied. **243 call sites across 19 controllers** (70 DML guards across 9 controllers + 173 SOQL guards across 18 classes). This implements the AND half of the reviewer's finding-resolution language. Example:

   ```
   DocGenFlsGuard.assertCreateable(job, new Set<String>{
       'Template__c', 'Status__c', 'Query_Condition__c'
   });
   /* code-analyzer-suppress ApexFlsViolation */
   Database.insert(job, AccessLevel.SYSTEM_MODE);
   ```

3. **Actual SOQL / DML uses `SYSTEM_MODE`** behind the two gates, with `/* code-analyzer-suppress ApexFlsViolation */` and inline justification. SYSTEM_MODE is required because USER_MODE strict-FLS strips package-namespaced custom fields (`Query_Config__c`, `Header_Html__c`, `Content_Version_Id__c`, etc.) when subscriber admin profiles haven't been granted FLS individually on each new render-config field across releases. The package-build scratch org reproduces this with `No such column 'Query_Config__c'` errors that break ~100 tests.

4. **Standard objects** (`ContentVersion`, `ContentDocumentLink`, `User`, `OrgWideEmailAddress`, etc.) use **`WITH USER_MODE`** throughout — no namespaced FLS issue, no need for DocGenFlsGuard either (platform-enforced FLS).

5. **Guest signing paths** retain `SYSTEM_MODE` (guests structurally cannot have DocGen CRUD by design — granting it would create the cross-tenant data-exposure vulnerability the reviewer was concerned about) but route through the `DocGenSignatureGuestSecurity` helper class (shipped in v2.0) that centralizes the Schema-CRUD describe checks + token-bound capability validation + per-operation field allowlists at every guest entry point, plus the v2.1.0 `DocGenFlsGuard` per-field guards at each guest DML/SOQL site.

---

6. Merge Tag Reference

6.1 Field tags

```
{Name}                          — simple field on base object
{Account.Owner.Email}           — dot-walk lookup chain
{Amount | currency}             — formatted field
{CloseDate | date:"MMM d, yyyy"} — date formatting
```

6.2 Loops

```
{#Contacts}
  {FirstName} {LastName} — {Email}
{/Contacts}
```

Loops over child relationships. Nested loops and junction loops are supported via the V3 query tree.

6.3 Conditionals

```
{?Amount > 10000}
  VIP customer — includes executive summary.
{/?}
```

6.4 Images

```
{%LogoField}
```

Resolves to a `ContentVersion` on the record. On the PDF path, only `Id, FileExtension` are queried — `VersionData` is deliberately excluded to stay under heap limits.

6.5 Barcodes

```
{$Code128:Account.AccountNumber}
{$QR:Id}
```

Rendered in-Apex by `BarcodeGenerator` and embedded as images.

6.6 Signature placeholders

```
{@Signature_Buyer}
{@Signature_Seller}
```

Preserved through `processXml()` during the ordinary merge pass (any tag starting with `@` is skipped) and replaced with the signer's typed name during signature finalization.

---

7. Query Configuration Formats

`DocGen_Saved_Query__c.Query_Config__c` (a 32 KB LongTextArea) supports three formats. The retriever auto-detects.

7.1 V1 — Legacy flat string

```
Name, Industry, (SELECT FirstName, LastName FROM Contacts)
```

7.2 V2 — Flat JSON with junctions

```
{
    "v": 2,
    "baseObject": "Opportunity",
    "baseFields": ["Name"],
    "parentFields": ["Account.Name"],
    "children": [{ "rel": "OpportunityLineItems", "fields":
        ["Name"] }],
    "junctions": [
        {
            "junctionRel": "OpportunityContactRoles",
            "targetObject": "Contact",
            "targetIdField": "ContactId",
            "targetFields": ["FirstName"]
        }
    ]
}
```

7.3 V3 — Query tree (multi-object, any depth)

```
{
    "v": 3,
```

```
    "root": "Account",
    "nodes": [
        {
            "id": "n0",
            "object": "Account",
            "fields": ["Name"],
            "parentFields": ["Owner.Name"],
            "parentNode": null,
            "lookupField": null,
            "relationshipName": null
        },
        {
            "id": "n1",
            "object": "Contact",
            "fields": ["FirstName"],
            "parentFields": [],
            "parentNode": "n0",
            "lookupField": "AccountId",
            "relationshipName": "Contacts"
        },
        {
            "id": "n2",
            "object": "Opportunity",
            "fields": ["Name", "Amount"],
            "parentFields": [],
            "parentNode": "n0",
            "lookupField": "AccountId",
            "relationshipName": "Opportunities"
        }
    ]
}
```

Each node becomes one SOQL query; results are stitched into the parent's data map via `lookupField`. The visual V3 builder is `docGenColumnBuilder` + `docGenTreeBuilder`.

All three formats are backwards compatible — existing templates continue to work after upgrades.

---

8. Heap and Governor-Limit Strategy

Document generation is heap-sensitive. DocGen v2.0 uses three complementary strategies:

1. **Pre-decomposed template parts.** When a template version is saved, `DocGenService` extracts each XML part (`document.xml`, `styles.xml`, etc.) and saves it as its own `ContentVersion`. At generation time, `tryMergeFromPreDecomposed()` loads only the parts it needs — no base64 decode, no ZIP decompression — yielding ~75% heap savings on the PDF path.
2. **Zero-heap image pipeline.** Template images are extracted to committed `ContentVersions` at save time. At generation time, `buildPdfImageMap()`

queries only `Id, FileExtension` and emits relative Shepherd URLs. Image bytes never transit Apex heap on the PDF path.
3. **Client-side ZIP assembly.** For large DOCX outputs, XML parts are returned to the browser; the browser fetches each image as a separate `@AuraEnabled` call (each gets a fresh 6 MB heap); `docGenZipWriter.js` assembles the ZIP locally.

The result: unlimited-size PDFs with many images, and unlimited-size DOCX outputs (for the Download path; "Save to Record" is capped by the Aura 4 MB payload ceiling).

---

9. Installation and First-Run Setup

1. **Install the managed package** in your production or sandbox org.
2. **Enable the Release Update** "Use the Visualforce PDF Rendering Service for `Blob.toPdf()` Invocations" (Setup → Release Updates). This is required for the PDF path.
3. **Assign permission sets:**
   - `DocGen_Admin` — admins.
   - `DocGen_User` — end users who should be able to generate documents.
4. **Add the `docGenRunner` component** to any record page layout where users should see a "Generate" button.
5. **Open the DocGen app** (App Launcher → DocGen) and follow the Setup Wizard in the Command Hub.

9.1 Optional: enable signatures

1. Create a **Salesforce Site** that publishes `DocGenSignature.page`.
2. Assign the `DocGen_Guest_Signature` permission set to the site's guest user.
3. Set **Email Deliverability** (Setup → Email Deliverability) to **All Email**.
4. Configure branding, return URL, and optionally an **Org-Wide Email Address** in Command Hub → Signatures tab.

---

10. Usage Walkthroughs

10.1 Creating a Template

1. Command Hub → **Templates** → **Create New**.
2. Name the template and pick the base sObject (e.g., `Account`).
3. Build the query using the visual builder (tab per object, tree preview) or paste manual SOQL.
4. Upload the `.docx` / `.xlsx` / `.pptx` file containing merge tags.
5. Save. The package extracts and commits template images; a new active `DocGen_Template_Version__c` record is created.

10.2 Generating a Single Document

1. Open a record page that has `docGenRunner`.
2. Pick a template, choose **Download** or **Save to Record**, pick the output format.
3. Click **Generate**.
4. The document downloads (or appears in the record's Files related list).

10.3 Bulk Generation

1. Command Hub → **Bulk Generate**.
2. Pick a template and a saved query (or build one inline).
3. Review the estimated record count and heap projection.
4. Launch the job. Progress is live-polled.
5. On completion, each record receives a `ContentVersion` with the generated document.

10.4 Sending a Signature Request (Signatures v3)

1. Open a record page with the `docGenSignatureSender` component.
2. Pick a template that contains `{@Signature_Role}` placeholders.
3. Add one signer per role (name + email). Optionally configure **sequential vs parallel** order and per-signer **PIN bypass / in-person** mode.
4. (Optional) For guided placements, drop per-tag positions onto the template preview to create `DocGen_Signature_Placement__c` records so each signer is walked through their tags in order.
5. Click **Send**. Each signer receives a branded email with a unique, time-limited link (or, for in-person mode, the admin hands the device directly to the signer).
6. Track progress in the signature request related list; completed PDFs appear in the record's Files list with the embedded Electronic Signature Certificate.

10.5 Verifying a Signed Document

1. Visit the verification URL printed on the signature certificate (`DocGenVerify.page`).
2. Drop the PDF onto the page.
3. The browser computes a SHA-256 hash **locally** (the file is never uploaded) and compares it to the audit record.
4. A green banner confirms integrity, or a red banner indicates the file has been modified. v2.0 returns the **complete** audit trail for every signer on a multi-signer PDF (v1.56 LIMIT-1 bug fixed — see §13).

10.6 Flow Integration

- `DocGenFlowAction` — generate a document for one record.
- `DocGenBulkFlowAction` — kick off a bulk job from a Flow.
- `DocGenGiantQueryFlowAction` — start a giant-query job.
- `DocGenSignatureFlowAction` — create a DocGen signature request from a Flow.

All four are registered as invocable actions and appear in the Flow Builder action picker under the "Document Generation" category.

10.7 Automating Signature Requests from Flow

A typical end-to-end automation pattern with `DocGenSignatureFlowAction`:

1. **Trigger:** Record-triggered Flow on an Opportunity (or Contract, Quote, custom object) — fires when a status changes to "Ready for Signature".
2. **Build signer collections:** Use Flow formula resources or loops to populate `signerNames`, `signerEmails`, and optionally `signerRoles` / `signerContactIds` text collections. Role names must match the `{@Signature_<Role>}` placeholders in the template.
3. **Invoke the action:** `DocGen: Create Signature Request` with the template Id, the triggering record Id, and the signer collections. Leave `Send Branded Emails` unset (defaults to **false**) so Flow owns the notification.
4. **Notify signers:** Loop over the returned `signerUrls` collection. For each signer, either:
   - Use Flow's **Send Email Action** with a custom template body that includes `{!currentSignerUrl}`, or
   - Call a custom HTTP-callout invocable to post to Slack/Teams/Chatter, or
   - Update the triggering record with the first signing link for an internal preview.
5. **Track state:** Update the triggering record with the returned `signatureRequestId` so you can report on outstanding signature requests and detect completion via the `DocGen_Signature_PDF__e` platform event trigger path.

Alternative: set `Send Branded Emails = true` to have the package send its built-in branded invitation emails, identical to the LWC Sender component's behavior. Use this when you want to automate request creation without writing custom email templates.

---

11. Known Limits and Guardrails

Area	Limit
Apex heap	6 MB sync / 12 MB async. DocGen uses pre-decomposed parts + zero-heap images + client-side ZIP to stay under.
`@AuraEnabled` payload	Aura framework caps at ~4 MB. Affects "Save to Record" for client-assembled DOCX — falls back to server ZIP path.
PDF fonts	`Blob.toPdf()` supports only Helvetica, Times, Courier, Arial Unicode MS. Custom fonts are a platform limitation.
Bulk job size	Limited by Batchable chunking and DML governor limits; `DocGen_Job__c` tracks per-record failures.
Signer session	48-hour token, 10-minute PIN, 3-attempt lockout.

---

12. Testing and Release Validation

Every release must pass three checks before shipping:

1. **End-to-end Apex suite** — 11 anonymous scripts in `scripts/e2e-*.apex` (e2e-01 through e2e-08 plus four `e2e-07-syntax*` variants) covering permissions, template CRUD, PDF generation, DOCX generation, bulk, signatures, merge-tag syntax, and cleanup. Each script prints `PASS: N FAIL: 0`.
2. **Apex test suite** — **1,449 tests, 76% org-wide coverage**, `sf apex run test --test-level RunLocalTests`.
3. **Code Analyzer** — `sf code-analyzer run --rule-selector Security --rule-selector AppExchange`. Must show **0 Critical / 0 High / 0 Moderate**. (38 documented false positives — `pmd:ProtectSensitiveData` + `pmd:AvoidLwcBubblesComposedTrue` — are disabled at the rule level in `code-analyzer.yml` with full structural justification.)

Per-release regression focus lives in the feature-area-specific e2e script (e.g., new merge-tag syntaxes must get a `processXmlForTest()` assertion in one of the `e2e-07-syntax*` scripts).

---

13. v2.0 → v2.1.0 Re-submission Highlights

This package version closes all 30 findings from the AppExchange security review of the v1.56 listing. Per-finding map: `SECURITY_REVIEW_RESPONSE_v2.md` in this folder. High-level summary:

- **4 clickjacking findings (v2.0)** → SLDS `slds-is-absolute` utility-class swap across 5 LWC bundles (`docGenAdmin`, `docGenAuthenticator`, `docGenBulkRunner`, `docGenQueryBuilder`, plus `docGenColumnBuilder` proactively).
- **26 CRUD/FLS findings (v2.0 object-level half)** → object-level Schema-CRUD gate at every admin entry point + SYSTEM_MODE on the actual operation. Guest endpoints gated by the `DocGenSignatureGuestSecurity` helper.
- **+1 in-flight bug fix (v2.0)** to `DocGenAuthenticatorController.verifyDocument` — now returns `List<VerificationResult>` so a multi-signer PDF dropped on the verifier returns the complete audit trail for every signer (previously LIMIT 1 only showed one signer).
- **562 Checkmarx findings closed (v2.1.0 per-field half)** — FLS Create (118) + FLS Update (104) + USER_MODE Missing (340) — via 243 `DocGenFlsGuard` call sites across 19 controllers. Real runtime enforcement: each guard performs `Schema.SObjectField.getDescribe().is{Createable,Updateable,Accessible}()` per field in the operation's allowlist. 38 remaining Moderate findings (`pmd:ProtectSensitiveData` field-name pattern matches + `pmd:AvoidLwcBubblesComposedTrue` on the recursive docGenTreeNode LWC) are documented false positives suppressed at the rule level in `code-analyzer.yml` with full structural justification — the False Positive Report is honest about this distinction.

v2.0/v2.1.0 also rolls forward all feature work since v1.56 (~45 versions): V3 query trees with visual query builder, the chart engine (9 styles, pure-Apex PNG via CV pipeline), signature v3 (PIN second factor, multi-signer with sequential/parallel order, per-tag guided placements, in-person signing flow), HTML templates with embedded image extraction, giant-query batching for multi-million-row child datasets, watermarks, document title formatting, and more.

---

14. Related Documentation

- `DocGen_Solution_Architecture_and_Usage.md` — security-review companion (data flows, threat model, sharing model, controls).
- `SECURITY.md` — disclosure policy and design principles.
- `docs/code-analysis/violations.md` — Code Analyzer output.
- `CLAUDE.md` — engineering invariants for anyone modifying the codebase.
- `CHANGELOG.md` — release history.

Portwood Global Solutions — https://portwood.dev